CommitFuzz: A Commit-Level Risk Scoring and Visualization Framework

Md Rokonuzzaman Reza, Tania Khatun, Richard Jiang

CSC-7135: Software Engineering

Survey of Related Works

- Point-in-Time Static Analysis (e.g., Clang-Tidy):
  - **Focus**: Identifies general code smells and style violations in isolation.
  - **Limitation**: Lacks change-specific context and often flags mature, safe code as "risky" (high false positives).
- Fuzzing Visualization (e.g., VisFuzz, Cvfuzz):
  - **VisFuzz**: Monitors active fuzzing to help humans find real-time bottlenecks through live Call Graphs and CFGs.
  - **Cvfuzz**: Detects complexity vulnerabilities specifically in OpenCL kernels via pathological input generation.
  - It does not predict risky commits early.
- Our CommitFuzz Contribution: Unlike these, we use Differential CFGs and Complexity Heatmaps to predict risk before expensive dynamic testing begins (not just visualization or general static checking + it is an early risk prediction method)

High-Level Architectural Design of CommitFuzz

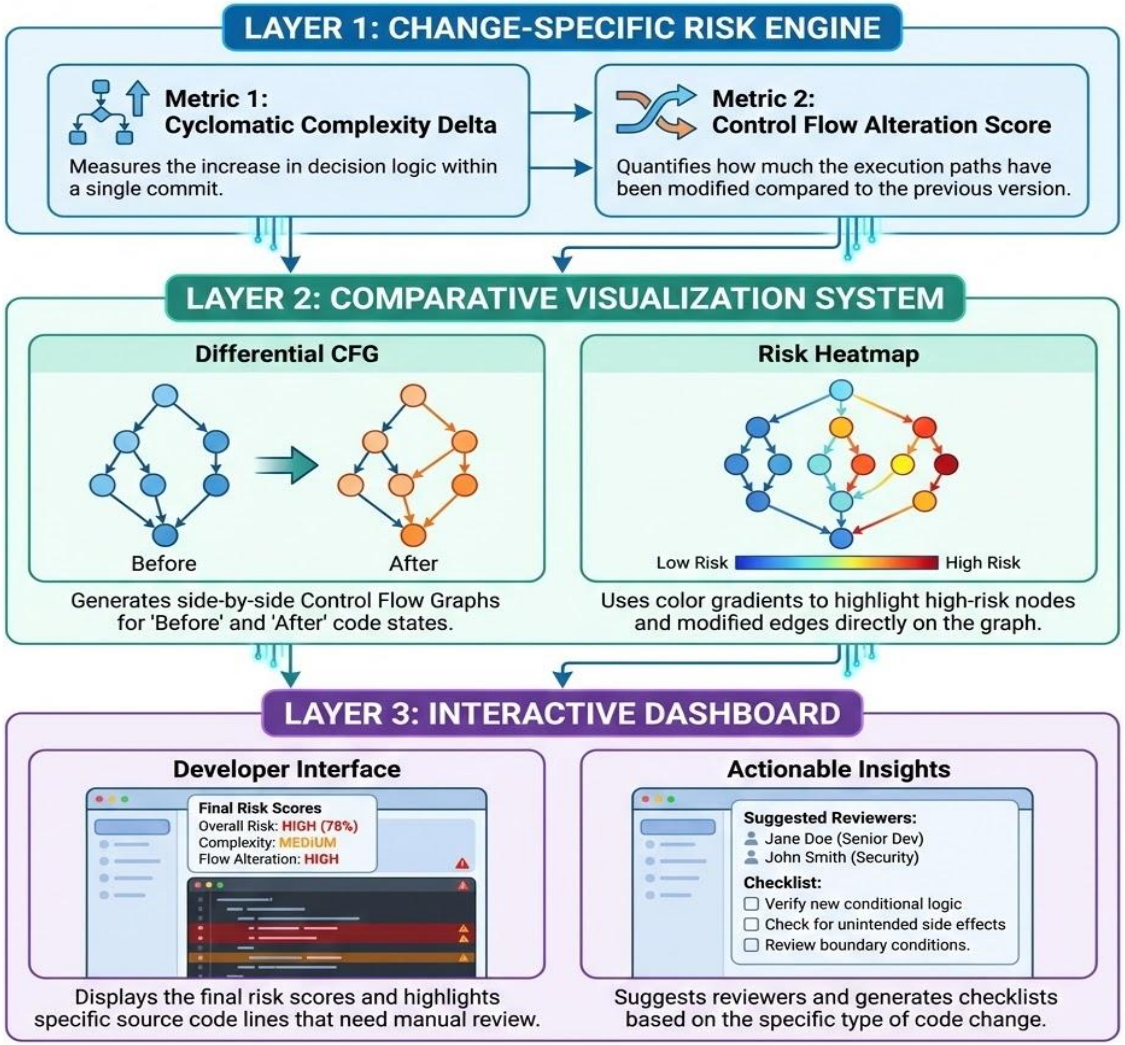

Fig-01: Architecture of CommitFuzz

High-Level Architectural Design

- Layer 1: Change-Specific Risk Engine
  - Metric 1: Cyclomatic Complexity Delta: Measures the increase in decision logic within a single commit.
  - Metric 2: Control Flow Alteration Score: Quantifies how much the execution paths have been modified compared to the previous version.
- Layer 2: Comparative Visualization System
  - Differential CFG: Generates side-by-side Control Flow Graphs for "Before" and "After" code states.
  - Risk Heatmap: Uses color gradients to highlight high-risk nodes and modified edges directly on the graph
- Layer 3: Interactive Dashboard
  - Developer Interface: Displays the final risk scores and highlights specific source code lines that need manual review.
  - Actionable Insights: Suggests reviewers and generates checklists based on the specific type of code change.

Professor Suggestion: Include Call Graph. Along with differential CFG we need to calculate differential call graph and integrate them together and do the rest of the part of the project. While calculating call graph we need to be careful about the dataset selection. Some repository will be very difficult for generating call graph.

Data Collection Pipeline

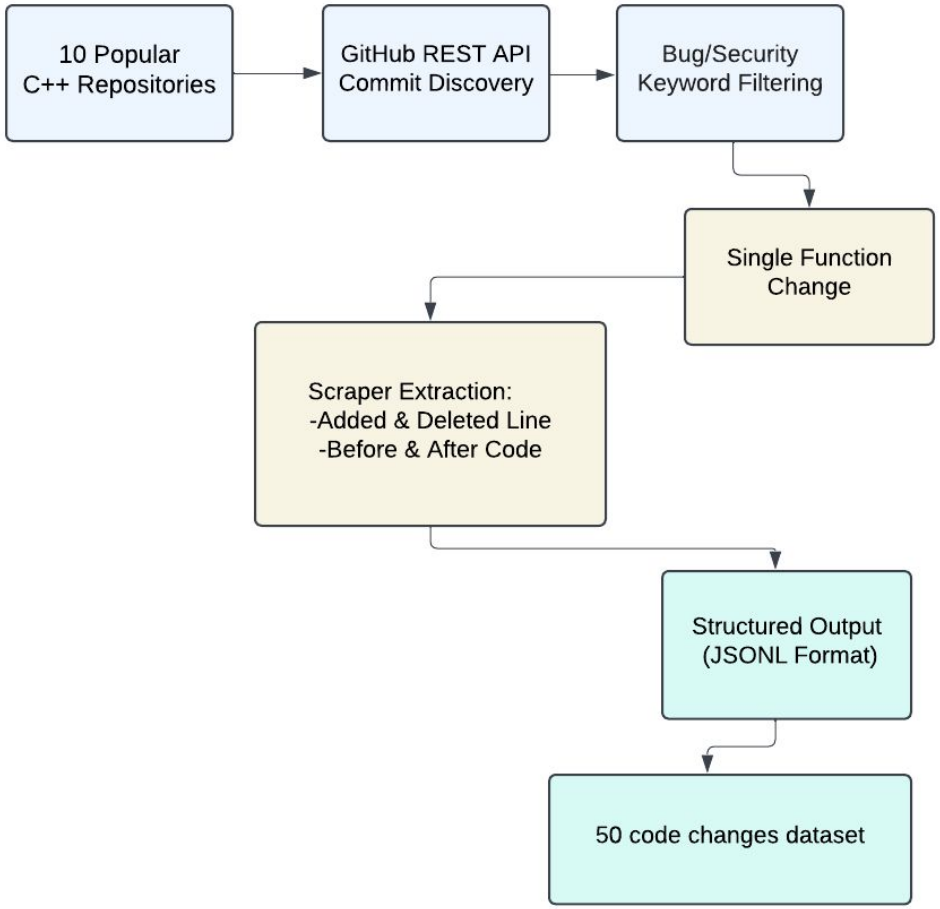

Fig-02: Dataset Collection Procedure

Progress Update – Data Collection Pipeline

- What Has Been Implemented:
  - Selected 10 popular C++ repositories
  - Automated commit discovery using GitHub REST API
  - Filtered commits using bug/security-related keywords
  - Built scraper to extract:
    - Single Function Change
    - Added and deleted lines
    - Before and After full code versions
- Stored structured outputs in JSONL format
- Pipeline complete → Currently collected 50 commits

Dataset

Sample data:

{"Commit title": "Add null check",
"Commit url": "https://github.com/test/repo/commit/123", "Only_addition_codes": "if (!ptr) return;",
"Only_deletion_codes": "", "Codes_without_addition_and_deletion": "void func(int *ptr) {\n}",
"Before_commit_codebase": "void func(int *ptr) {\n}", "After_commit_codebase": "void func(int *ptr) {\nif (!ptr) return;\n}"}

Repositories:

```
repos = [
  "FFmpeg/FFmpeg",
  "opencv/opencv",
  "nlohmann/json",
  "google/leveldb",
  "google/re2",
  "facebook/folly",
  "apache/arrow",
  "libjpeg-turbo/libjpeg-turbo",
  "cmu-sei/pharos",
  "boostorg/algorithm"]
```

Dataset:

https://github.com/EmonRezaBD/commitriskviz/blob/master/data/singleFuncDataset.jsonl

Questions?

Thank you for your attention.